FINAL PROJECT PROPOSAL

Database Technology: Event Management System (Eventra)



Student by :

Aliansky Winston Viando - 2902708224

Nizar Ramadhan - 2902725672

Renjiro Kunio Handoko- 2902740591

INTRODUCTION

1.1 Project Description

Eventra is the platform of selling tickets online for the event organizers in small and medium sized events in Jakarta such as orchestra, music community, and event committee campus. the main example that we used is using local orchestra that is provided in several concerts in every season in the rent building with the seat number

At this time, organizers like this sell tickets through google form, whatsapp chat and spreadsheet, then confirming transfers through the bank manually. These showing some problem such as:

- The same seat can be sold twice because no one record shows which seats are taken.
- Buyer details are copied into several spreadsheets, so they often don't match.
- Payments must be checked one by one against bank statements.
- Attendance at the door is tracked on paper, so there's no reliable record of who actually came.
- Discount codes, refunds, and waitlists are handled informally and are hard to audit.

This project designs and implements a relational database, with a connected user interface, that stores all of this information in one place. Organizers can create events, define venues and seating, set ticket types and prices, and issue vouchers. Customers can browse events, place orders, pay, and receive digital tickets. Staff can check attendees in at the venue.

1.2 Objectives

1. Design a normalized database (up to 3NF) to store events, tickets, orders, and payments without duplicate data.
2. Prevent overselling and double booking by limiting ticket quotas and seats per event.
3. Record the full ticket purchase process, from order to payment to ticket.
4. Provide simple reports, such as tickets sold and revenue per event.
5. Build a user interface connected to the MySQL database.

1.3 Target Users

User	Role
Admin	Manages the whole platform: verifies

	organizers, manages categories, and handles refunds.
Organizer	Creates events, sets up venues and ticket types, creates vouchers, and views their sales reports.
Staff	Scans tickets and checks attendees in on event day.
User / Attendee	Searches events, buys tickets, makes payments, and writes reviews after attending.

1.4 Key Business Processes

1. **Organizer Registration.** An organizer signs up, and the admin verifies their details before they can publish events.
2. **Event Setup.** The organizer creates an event, chooses a category and venue, and sets up ticket types (e.g., VIP, Regular, Student) with prices and quotas.
3. **Ticket Purchase.** The user picks an event, ticket type, quantity, and seat, and can apply for a voucher. The system creates the order and calculates the total price.
4. **Payment.** The user pays for the order. Once the payment succeeds, the system generates digital tickets, each with a unique code.
5. **Check-in.** On event day, staff scan the ticket code, and the system marks the ticket as used.
6. **Refunds and Reviews.** If an event is cancelled, the payment is refunded. After the event, attendees can leave a review.

1.4.1 Business Rules

1. Every user needs to register with an email that is not used yet. An organizer has to be verified by the admin first before they can post an event, and one organizer can make many events, but one event only belongs to one organizer.

2. Each event has one category and is held at one venue. One venue has many sections, each section has many seats, and one event can have more than one ticket type, like VIP, Regular, or Student, with its own price and quota.

3. Tickets cannot be sold more than the quota, one seat can only be booked once for the same event, and each ticket has its own unique code.

4. One user can make many orders, but one order is only for one event and can only use one valid voucher. The ticket is only given to the user after the payment is successful.

5. If the event is cancelled, the user can get a refund and the ticket will be cancelled. On the event day, staff scan the ticket and one ticket can only be scanned once. After the event, users who joined can give one review with a rating from 1 to 5.

1.5 Database Considerations

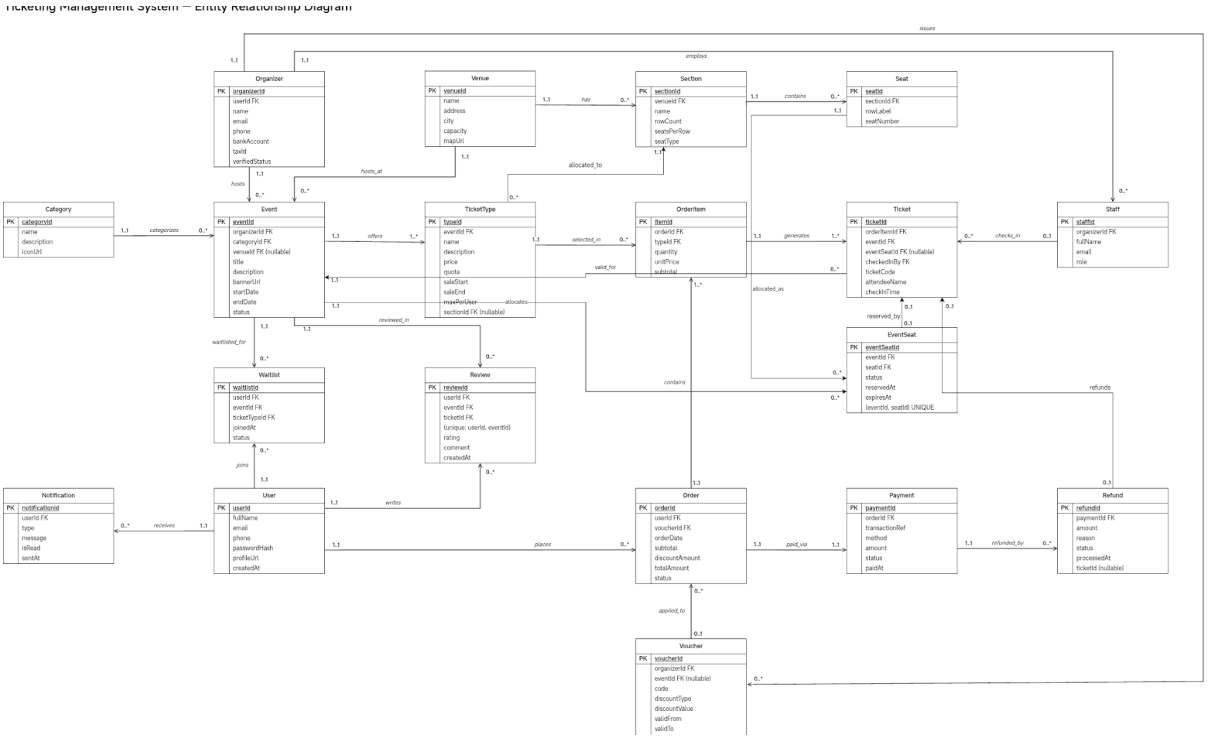
`**Data integrity.** Every table has a primary key, and relationships between tables are enforced with foreign keys.

- **No duplication.** User, organizer, and venue data is stored only once and referenced by ID.
- **No double booking.** A seat can only be booked once per event.
- **Constraints.** Emails, ticket codes, and voucher codes must be unique. Prices and quantities cannot be negative, and ratings must be between 1 and 5.
- **Security.** Passwords are stored as hashes, and each user can only access data that fits their role.

2.1 PROJECT TIMELINE

Week	Phase	Activities
Week 1–2	Planning	Choose the business scenario, identify users, and define business processes
Week 3–4	Database Design	Create the ERD and define entities, attributes, and relationships
Week 5	Normalization & Proposal	Normalize the data up to 3NF and submit the proposal
Week 6–8	Implementation	Create tables in MySQL, insert sample data, and write SQL queries
Week 9–11	UI Development & Testing	Build the user interface, connect it to MySQL, and test the system
Week 12–13	Final Report & Presentation	Finish the final report and present the project demo

Database Design



3.1 Entity-Relationship Diagram (ERD)

An Entity-Relationship Diagram (ERD) functions as a conceptual blueprint for a database architecture. It provides a visual schematic representing the primary entities within a system and their reciprocal associations. Database design utilizes ERDs to plan information storage prior to implementation, ensuring structural integrity, operational efficiency, and comprehensive support for application requirements.

Within the Ticketing Management System, the ERD organizes core platform data, encompassing user profiles, event administration, venue topology, ticket transactions, and financial records. The schema utilizes Primary Keys (PK) to uniquely identify individual records and Foreign Keys (FK) to establish cross-table references, thereby minimizing data redundancy.
The system comprises several core entity categories:

- Users and Organizers: Manage platform participants, including consumers and event hosts.
- Events, Venues, and Categories: Maintain specifications regarding event definitions, geographical locations, and seating configurations.
- Tickets and Orders: Administer transactional workflows, available ticket inventories, order histories, and generated admissions.

- Payments and Refunds: Ensure secure processing of financial transactions.
- Supporting Features: Incorporate ancillary modules such as waitlists, reviews, notifications, vouchers, staff management, and event-specific seat allocation.

3.2 Entities and Attributes

Prior to reviewing specific database relations, standard design terminology is defined as follows:

- Entity: A distinct object or concept targeted for data persistence.
- Attribute: A specific data property characterizing an entity.
- Primary Key (PK): An attribute that uniquely distinguishes each record within an entity.
- Foreign Key (FK): An attribute referencing a Primary Key in another entity to establish a relationship.

The entity catalog for the Ticketing Management System, aligned with the ERD architecture, is structured as follows:

1. Organizer

Purpose: Stores profiles of entities hosting and managing events.

Attributes:

- organizerId (PK): Uniquely identifies each organizer.
- name: Stores the name of the organizer or company.
- email: Stores the contact email.
- phone: Stores the telephone contact.
- bankAccount: Stores financial routing information for revenue payouts.
- taxId: Stores the tax identification number.
- verifiedStatus: Tracks platform approval and verification standing.

2. Venue

Purpose: Maintains physical locations designated for hosting events.

Attributes:

- venueId (PK): Uniquely identifies each venue.
- name: Stores the name of the establishment.
- address: Stores the street address.
- city: Stores the municipal location.
- capacity: Stores the maximum spatial occupancy limit.
- mapUrl: Stores an external hyperlink to spatial mapping data.

3. Section

Purpose: Segments venues into distinct seating sectors (e.g., VIP, Balcony, General Admission).
Attributes:

- sectionId (PK): Uniquely identifies each section.
- venueId (FK): Connects the section to its parent venue.
- name: Stores the section identifier.
- rowCount: Stores the total quantity of rows.
- seatsPerRow: Stores the number of seats per row.
- seatType: Specifies the seating model (reserved or open access).

4. Seat
Purpose: Represents individual physical seating units within a section.
Attributes:

- seatId (PK): Uniquely identifies each seat.
- sectionId (FK): Connects the seat to its parent section.
- rowLabel: Stores the row identifier designation.
- seatNumber: Stores the sequential seat number.
- status: Tracks operational availability or maintenance status.

5. Category
Purpose: Classifies events by thematic genre.
Attributes:

- categoryId (PK): Uniquely identifies each category.
- name: Stores the category title.
- description: Stores a summary explanation.
- iconUrl: Stores a link to a graphical representation asset.

6. Event
Purpose: Stores comprehensive configuration parameters for scheduled platform events.
Attributes:

- eventId (PK): Uniquely identifies each event.
- organizerId (FK): Connects the event to the hosting organizer.
- categoryId (FK): Connects the event to its thematic classification.
- venueId (FK): Connects the event to its physical location.
- title: Stores the event title.
- description: Stores detailed textual context.
- bannerUrl: Stores the promotional asset image link.
- startDate: Records the commencement timestamp.
- endDate: Records the conclusion timestamp.
- status: Tracks the lifecycle state (upcoming, ongoing, completed).

7. TicketType
Purpose: Defines ticketing tiers and associated pricing and access rules.
Attributes:

- typeId (PK): Uniquely identifies each ticket tier.
- eventId (FK): Connects the ticket type to its parent event.
- sectionId (FK) [nullable]: Optionally links the ticket type to a specific venue section.
- name: Stores the tier designation.
- description: Explains associated access privileges.
- price: Stores the unit cost.
- quota: Stores the inventory limit.
- saleStart: Records sales commencement timing.
- saleEnd: Records sales termination timing.
- maxPerUser: Restricts maximum acquisition limits per user.

8. OrderItem
Purpose: Acts as an associative junction entity linking orders to selected ticket types.
Attributes:

- itemId (PK): Uniquely identifies each order line item.
- orderId (FK): Connects the item to the primary transaction order.
- typeId (FK): Connects the item to the specific ticket type purchased.
- quantity: Stores the purchased quantity.
- unitPrice: Stores the locked unit price at transaction execution.
- subtotal: Stores the cumulative line-item cost.

9. EventSeat
Purpose: Tracks real-time spatial allocation, reservation locking, and availability status per event instance.
Attributes:

- eventSeatId (PK): Uniquely identifies each event-seat mapping instance.
- eventId (FK): Connects the seat status record to the target event.
- seatId (FK): Connects to the base physical seat entity.
- status: Tracks reservation states (available, locked, booked).
- reservedAt: Records temporary lock initiation timestamps.
- heldUntil: Records temporary reservation expiration timing.

10. Ticket
Purpose: Represents individual admission passes generated post-transaction.
Attributes:

- ticketId (PK): Uniquely identifies each issued ticket.
- orderItemId (FK): Connects the ticket to its originating order line item.
- eventSeatId (FK) [nullable]: Connects the ticket to a specific event-seat allocation.
- staffId (FK) [nullable]: Connects the ticket to the verification personnel who processed check-in.
- ticketCode: Stores unique scanning identifiers (barcode or QR string).

- attendeeName: Stores the designated ticket holder name.
- status: Tracks lifecycle validity (valid, used, cancelled).
- checkInTime: Records access validation timestamps.

11. Staff
Purpose: Manages operational personnel deployed by event organizers.
Attributes:
- staffId (PK): Uniquely identifies each staff member.
- organizerId (FK): Connects staff to the employing organizer.
- fullName: Stores the individual's full legal name.
- email: Stores contact email routing.
- role: Designates operational responsibilities (e.g., scanner, security).

12. Waitlist
Purpose: Provides queueing mechanisms for exhausted event or inventory pools.
Attributes:
- waitlistId (PK): Uniquely identifies each queue record.
- userId (FK): Connects the queue entry to the waiting user.
- eventId (FK): Connects the entry to the targeted event.
- ticketTypeId (FK) [nullable]: Optionally targets a specific ticket tier.
- joinedAt: Records queue entry timestamps.

status: Tracks queue progression standing.

13. Review
Purpose: Captures post-event ratings and qualitative user feedback.
Attributes:
- reviewId (PK): Uniquely identifies each review.
- userId (FK): Connects the review to the authoring user.
- eventId (FK): Connects the review to the evaluated event.
- rating: Stores quantitative rating metrics.
- comment: Stores qualitative textual feedback.
- createdAt: Records publication timestamps.

14. Notification
Purpose: Manages system-generated communications dispatched to platform users.
Attributes:
- notificationId (PK): Uniquely identifies each notification message.
- userId (FK): Connects the message to the recipient user.
- type: Classifies the notification category.
- message: Stores payload text content.
- isRead: Tracks engagement status.
- sentAt: Records transmission timestamps.

15. User

Purpose: Maintains registered customer account profiles.
Attributes:

- userId (PK): Uniquely identifies each user.
- fullName: Stores the user's legal name.
- email: Stores the primary electronic mail address.
- phone: Stores telephone contact information.
- passwordHash: Stores cryptographic hash strings for credential validation.
- profileUrl: Stores asset references for avatar images.
- createdAt: Records account creation timestamps.

16. Order
Purpose: Aggregates purchase line items into a unified transaction session.
Attributes:

- orderId (PK): Uniquely identifies each order session.
- userId (FK): Connects the transaction to the purchasing user.
- voucherId (FK) [nullable]: Connects the order to applied promotional instruments.
- orderDate: Records transaction placement timestamps.
- subTotal: Stores pre-discount financial totals.
- discountAmount: Stores deductions applied via vouchers.
- totalAmount: Stores the final payable sum.
- status: Tracks fulfillment states (pending, paid, cancelled).

17. Payment
Purpose: Records financial execution metrics for completed transactional orders.
Attributes:

- paymentId (PK): Uniquely identifies each payment record.
- orderId (FK): Connects payment metrics to the corresponding order session.
- transactionRef: Stores external gateway receipt identifiers.
- method: Stores the financial instrument classification used.
- amount: Stores successfully transferred capital sums.
- status: Tracks processing outcomes.
- paidAt: Records capital settlement timestamps.

18. Refund
Purpose: Administers capital returns associated with reversed or cancelled transactions.
Attributes:

- refundId (PK): Uniquely identifies each refund record.
- paymentId (FK): Connects the refund to the original payment transaction.
- amount: Stores the reimbursed capital sum.
- reason: Documents the justification for reimbursement.
- status: Tracks processing progression.
- processedAt: Records settlement execution timestamps.

19. Voucher
Purpose: Controls promotional discount structures established by organizers.

Attributes:

- voucherId (PK): Uniquely identifies each promotional instrument.
- organizerId (FK): Connects the voucher to the issuing organizer.
- code: Stores alphanumeric redemption text tokens.
- discountType: Specifies structural discount calculation rules.
- discountValue: Stores numerical discount magnitudes.
- validFrom: Records activation parameters.
- validTo: Records expiration parameters.
- maxUses: Enforces redemption aggregate constraints.

Entity Workflow Integration

- Event and Venue Management (Organizer, Category, Event, Venue, Section, Seat, EventSeat, Staff): Establishes structural foundations, environmental topology, spatial allocation locks via EventSeat, and operational personnel deployment.
- Ticket Management (TicketType, OrderItem, Ticket): Governs inventory availability, order line item mapping, and admission generation tied to precise seat structures.
- Customer Activity (User, Waitlist, Review, Notification): Controls user interaction lifecycles, alert systems, queue management, and feedback collection.
- Order and Payment Management (Order, Payment, Refund, Voucher): Manages transactional financial flows, cryptographic validation references, promotional discounts, and capital reversals.

3.3 Relationships and Cardinality

In database design, a relationship is a rule that explains how two separate entities interact with each other. For example, the rule that "a user places an order" is a relationship between the User entity and the Order entity.

To explain these relationships precisely, we use cardinality. Cardinality is a way of counting the minimum and maximum number of connections allowed between two entities.

- 1..1 means exactly one.
- 0..1 means zero or one (it is optional).
- 0..* means zero, one, or many (unlimited).
- 1..* means at least one, or many.

These cardinalities create different types of database relationships:

- One-to-One (1:1): One record in a table connects to exactly one record in another table. (Example: One Order connects to exactly one Payment).
- One-to-Many (1:N): One record connects to many records in another table. (Example: One Organizer can host many Events).
- Many-to-Many (M:N): Many records in one table connect to many records in another table. (Our diagram breaks these down using middle tables, like OrderItem bridging Orders and TicketTypes).

NORMALIZATION

The complete normalization process, including the unnormalized data and the transformation into 1NF, 2NF, and 3NF, is documented in the following Google Sheets link.

Google Sheets:
https://docs.google.com/spreadsheets/d/1vLGuv-omvrFOuTLLr_qNeuSYZRYEOyT7/edit

The first step in the normalization of data is to use sample data that represents the users, events, organizers, venues, ticket types, orders, tickets, payments, and vouchers. By separating the attributes based on their dependencies, the database is designed to store each type of information in an appropriate table while maintaining relationships through primary keys (PK) and foreign keys (FK).

4.1 Data Normalization Process

UNF (Unnormalized Form)

The initial dataset is designed to represent the information involved in purchasing the event tickets. It contains the order details, user's information, event's information, organizer's details, venue information, ticket items, individual tickets, voucher information, and payment details.

In the UNF, several attributes have repeating groups. For example, one single order can contain several other order items or tickets, while vouchers may also be included in that same order. These groups make the data more difficult to maintain and query because of the multiple values that can be stored within one field.

If an order has multiple tickets, placing all that information in a single record will make it difficult to tell individual tickets, update the statuses, or manage quantities precisely.

1NF (1st Normal Form)

The UNF is made into a 1NF by ensuring that each attribute has a single, atomic value and that repeating groups are deleted. Each record must represent an identifiable instance of the data being stored.

For example, instead of keeping multiple tickets in one field, each ticket is shown as an individual record with its own ticket identifier, type, and seat information. Similarly, order items and voucher information has to be represented in an easy to understand manner rather than as multiple values within a single field.

The 1NF dataset retains the information required to represent orders, users, events, organizers, venues, tickets, and payments. However, some information is still repeated across

multiple records. For example, several orders associated with the same user may repeat the user's name, email address, and phone number. Likewise, multiple orders for the same vent may repeat the event name, venue details, and organizer information.

To prepare for the next normalization stage, the initial composite keyu for the order-item data is identified as the combination of orderID and ticketTypeID. Under the assumption that each order contains at most one order-item record for each ticket type, this combination uniquely identifies an order item.

Although the data satisfies 1NF, the repeated information and partial dependencies indicate that further decomposition is necessary.

2NF (2nd Normal Form)

Second Normal Form requires a table to satisfy 1NF and ensures that every non-key attribute depends on the entire candidate key rather than only part of a composite key.

During this stage the order-item data is separated from the general order information. The initial order-item structure uses the composite key (orderID, ticketTypeID) to identify the combination of an order and a ticket type.

The Normalization process identifies the following dependencies:

- orderID determines the order date, order status, subtotal, discount amount, and total amount
- userID determines the user's account information including their personal information.
- The combination of orderID and ticketTypeID identifies the corresponding order item, including its quantity and unit price.

Separating these attributes prevents user and order information from being unnecessarily repeated for every ticket type included in a purchase.

This decomposition reduces redundancy and prevents update anomalies. Changing a user;s contact information or an order's status can be handled in the appropriate table without requiring the same information to be updated across multiple order item records.

3NF (3rd Normal Form)

Third Normal Form requires a table to satisfy 2NF and ensures that non-key attributes depend on the key, the whole key, and nothing but the key. In particular, non-key attributes should not depend transitively on a primary key through another non-key attribute.

In this stage, the database is decomposed into separate tables based on the functional dependencies identified in the data. Each table represents a particular entity or business concept, and its non-key attributes describe that entity.

The final database design contains the following 18 tables:

1. Organizer – Stores organizer information, including name, email, phone, bank account, and verification status.
2. Venue – Stores venue information, including name, address, city, capacity, and map URL.
3. Section – Stores venue sections and their seating configuration.
4. Seat – Stores individual seat identifiers and their section, row, and seat number.
5. Category – Stores event categories, descriptions, and icon URLs.
6. Event – Stores event details and references the corresponding organizer, category, and venue.
7. TicketType – Stores ticket options, descriptions, prices, quotas, sale periods, and purchase limits for an event.
8. User – Stores user account and profile information.
9. Staff – Stores staff information and their association with an organizer.
10. Order – Stores the user associated with a purchase, order date, subtotal, discount amount, total amount, and status.
11. OrderItem – Stores the ticket type, quantity, unit price, and subtotal for each item in an order.
12. Ticket – Stores individual ticket records and their associated order, user, event, ticket type, seat, and status.
13. Payment – Stores payment methods, amounts, and payment statuses associated with orders.
14. Refund – Stores refund information, including the associated payment, reason, status, and processing date.
15. Voucher – Stores voucher codes, discount rules, validity periods, usage limits, and the associated organizer.
16. Review – Stores user reviews of events, including ratings, comments, and creation dates.
17. Waitlist – Stores users waiting for events, including their joining dates and statuses.
18. Notification – Stores notifications sent to users, including message content, notification type, read status, and sent date.

The decomposition separates information that would otherwise be repeated. For example, venue details are stored in the Venue table rather than repeated for every event held at the same location. Similarly, event categories are stored separately from event records, and organizer details are separated from the events they manage.

TicketType is also separated from Ticket. This allows the system to manage ticket options and their prices at the event level while maintaining individual ticket records for purchases

and attendance. OrderItem separates the items included in a purchase from the general information stored in Order.

The final schema uses individual identifiers, such as eventID, userID, venueID, and orderID, as primary keys for their respective tables. OrderItem uses orderItemID in the final 3NF design, while its orderID and ticketTypeID attributes reference the associated Order and TicketType records.

The final design therefore reduces unnecessary duplication, improves data maintenance, and establishes a structured foundation for implementing the Event Management System.

4.2 Keys and Referential Integrity

(PK) = Primary Key

(FK) = Foreign Key

Organizer:

PK: organizerID

Venue:

PK: venueID

Section:

PK: sectionID

FK: venueID

Seat:

PK: seatID

FK: sectionID

Category:

PK: categoryID

Event:

PK: eventID

FK: organizerID, categoryID, venueID

TicketType:

PK: ticketTypeID

FK: eventID

User:

PK: userID

Staff:

PK: staffID

FK: organizerID

Order:

PK: orderID

FK: userID

OrderItem:

PK: orderItemID

FK: orderID, ticketTypeID

Ticket:

PK: ticketID

FK: orderID, userID, eventID, ticketTypeID, seatID

Payment:

PK: paymentID

FK: orderID

Refund:

PK: refundID

FK: paymentID

Voucher:

PK: voucherID

FK: organizerID

Review:

PK: reviewID

FK: userID, eventID

Waitlist:

PK: waitlistID

FK: userID, eventID

Notification:

PK: notificationID

FK: userID